

Failure Classification in DBMS

Overview

In a database context, **failure** is defined as the system's inability to execute a specified transaction or the loss of data from the database. Because a DBMS is vulnerable to various disruptions, each type of failure requires a specific management and recovery strategy.

Common Causes of Failure:

- Network failures or system crashes.
- Natural disasters or power outages.
- Human factors (carelessness or intentional sabotage/corruption).
- Software bugs and resource unavailability.

1. Transaction Failure

A transaction failure occurs when a transaction cannot finish its execution or reaches a point where it can no longer proceed. These are generally categorized into two types:

Error Type	Description	Examples
Logical Error	Occurs when the transaction cannot complete due to internal code issues or logic faults.	Incorrect code, "Division by Zero" errors.
System Error	The DBMS terminates an active transaction because it cannot proceed with execution.	Deadlocks, resource exhaustion (CPU/RAM).

2. System Crash

A system crash is typically a hardware or software breakdown that causes the system to stop abruptly.

- **Characteristics:** Often referred to as "**Soft Failures.**"
- **Impact:** Primarily affects data stored in **volatile memory** (RAM).
- **The Fail-Stop Assumption:** This is the critical assumption that a system crash **does not** damage data stored in non-volatile storage (Hard Disks/SSDs).
- **Common Causes:** OS errors, main memory crashes, or sudden power loss.

3. Data-Transfer Failure (Disk Failure)

These failures occur during data-transfer operations, leading to the loss of content physically stored on the disk.

- **Common Causes:** * Disk head crashes.
 - Formation of **bad sectors**.
 - Read-write errors or disk unreachability.
- **Recovery Strategy:** * **Redundancy:** Using backup copies stored on secondary tapes or disks.
 - **Best Practice:** Frequent data backups are essential to minimize data loss during a disk failure.

What is database backup and recovery?

Database backup and recovery techniques refer to the process of making copies of your data and restoring it when things go wrong. These techniques ensure business continuity, protect against loss, and keep your systems running smoothly.

Difference between backup and recovery

Data backup and recovery are two sides of the same coin, but they serve very different purposes.

Backup is all about preparation. You proactively create copies of your data before anything goes wrong, often on a schedule, using tools that offer automated data replication, snapshot utilities, or cloud storage services. The goal is to preserve data in a safe, restorable state.

Recovery is more reactive. It kicks in after a problem like a crash, corruption, or accidental deletion occurs, using a backup to restore systems to a functional state. Recovery tools and processes include rollbacks, rollforwards, point-in-time restores, and full system restores.

Need for Backup

- Protects against data loss
- Helps recover after failure
- Supports business continuity
- Prevents downtime

Example:

Bank server crashes → backup helps restore customer balances.

Types of data backup techniques

Not every situation calls for a full copy of your database. There are several types of database backups, each designed to suit different scenarios. Here's how the main approaches compare.

Full database backups

A full database backup creates a complete copy of your entire database. It's thorough and straightforward but also time-consuming and storage-heavy. While full backups are great for clean restore points, you'll likely rack up quite the cloud bill if you create them all the time.

Example:

All student records copied every Sunday.

Incremental database backup

Incremental backups only save data that changed since the last backup of any type. This makes them fast, lightweight, and ideal for frequent backup schedules. It's the same principle as iCloud only backing up your new photos rather than your entire camera roll. The downside of incremental backups is that recovery means painstakingly stitching together multiple smaller files.

Example:

Monday: full backup

Tuesday: only new records saved

Differential database backup

Differential backups record everything that changed since the last full backup. They take up more space than incremental backups, but it's quicker to action a recovery, since you only need two files: the last full and the last differential. It's a happy medium for teams that want efficiency without complex restore chains.

Example:

Sunday full backup

Monday changes saved

Tuesday saves Monday + Tuesday changes

Recovery in DBMS

A Database Management System must protect data from failures such as system crashes, power failures, software errors, or hardware faults. During transaction execution, some changes may be completed while others may remain incomplete. If failure occurs in the middle of execution, the database can enter an inconsistent state.

To bring the database back to a correct and consistent state, the DBMS uses **recovery techniques**. Two important concepts used in recovery are:

- **Log-based recovery**
- **Checkpoint mechanism**

These techniques help the DBMS recover committed transactions and undo incomplete transactions.

1. Log-Based Recovery

Introduction

Log-based recovery is a recovery method in which the DBMS keeps a record of all transaction activities in a special file called the log.

The log stores information about:

- start of a transaction
- read and write operations
- old values of data items
- new values of data items
- commit of transaction
- abort of transaction

This log is used after a failure to determine:

- which transactions should be **redone**
- which transactions should be **undone**

Thus, log-based recovery is one of the most important techniques for maintaining database consistency.

Typical log records are:

- <Ti start>
- <Ti, X, old_value, new_value>
- <Ti commit>
- <Ti abort>

Where:

- Ti = transaction identifier
- X = data item modified
- old_value = value before update
- new_value = value after update

Write-Ahead Logging (WAL)

An important rule in log-based recovery is **Write-Ahead Logging**.

Rule:

The log record must be written to stable storage **before** the actual data item is written to the database.

This means:

- first update is recorded in log
- then actual database page is updated

This ensures that even if failure occurs after writing the log but before writing the data, the system can still recover.

Types of Log-Based Recovery

There are mainly two approaches:

- **Deferred database modification**
- **Immediate database modification**

1. Deferred Database Modification

In **deferred modification**, the actual database is not updated until the transaction reaches its commit point.

The updates are recorded only in the log during execution. After the transaction commits, the changes are applied to the database.

This approach is also called **No-Undo/Redo** in some contexts.

How it Works

1. Transaction starts
2. Update operations are recorded in the log
3. Actual database is not changed immediately
4. If transaction commits, updates are applied to database
5. If system crashes before commit, no database change was made, so no undo is required
6. If crash occurs after commit but before all updates are written, redo may be needed

Example

Suppose transaction **T1** transfers money.

Log records:

- <T1 start>
- <T1, A, 5000, 4000>
- <T1, B, 3000, 4000>
- <T1 commit>

If crash occurs before commit:

- database not updated yet
- no undo needed

If crash occurs after commit but before data is fully written:

- redo T1

Advantages of Deferred Modification

- simpler recovery logic
- no need to undo uncommitted transactions
- safer because database is updated only after commit

Disadvantages

- updates are delayed
- may reduce speed in some systems
- committed results may not appear immediately

Immediate Database Modification

In **immediate modification**, the database can be updated before the transaction commits.

This means changes may reach the database while the transaction is still active.

Hence, if failure occurs, the system may need:

- **undo** for uncommitted transactions
- **redo** for committed transactions

This is why immediate modification is often called **Undo/Redo** based recovery.

How it Works

1. Transaction starts
2. Log records are written before data is updated
3. Database may be updated before commit
4. If transaction commits and crash occurs, redo may be needed
5. If transaction does not commit and crash occurs, undo is needed

Example

Transaction T2 updates account balance.

Log:

- <T2 start>
- <T2, A, 1000, 800>
- <T2, B, 2000, 2200>

Suppose the system crashes before <T2 commit>

Then:

- database may already contain partial updates
- since transaction did not commit, these updates must be undone

Suppose crash occurs after commit:

- committed updates may need redo if not fully saved

Advantages of Immediate Modification

- allows faster buffer and page handling
- suitable for practical database systems
- more flexible

Disadvantages

- recovery is more complex
- both undo and redo may be needed
- partial updates may exist in database before commit

Using the Log to Redo and Undo Transactions

Example: Banking Transactions

```
T0: Transfer $50 from A to B      T1: Withdraw $100 from C
  read(A); A := A - 50;          read(C); C := C - 100;
  write(A); read(B);             write(C).
  B := B + 50; write(B).
```

Initial values: A = \$1000, B = \$2000, C = \$700

Log records:

```
<T0 start>
<T0, A, 1000, 950>
<T0, B, 2000, 2050>
<T0 commit>
<T1 start>
<T1, C, 700, 600>
<T1 commit>
```

Redo and Undo Procedures

redo(Ti): Sets the value of all data items updated by Ti to the NEW values from the log records.

undo(Ti): Restores the value of all data items updated by Ti to the OLD values. Also writes redo-only log records and finally a <Ti abort> record.

Recovery Decision Rules

Undo Ti if: The log contains <Ti start> but does NOT contain <Ti commit> or <Ti abort>.

Redo Ti if: The log contains <Ti start> AND either <Ti commit> or <Ti abort>.

Example: Three Crash Scenarios

Case (a): Crash after <T0, B, 2000, 2050> is logged (before T0 commit)

-> undo(T0): A restored to \$1000, B restored to \$2000

Case (b): Crash after <T1, C, 700, 600> is logged (T0 committed, T1 not)

-> redo(T0): A=\$950, B=\$2050

-> undo(T1): C restored to \$700

Case (c): Crash after <T1 commit> is logged (both committed)

-> redo(T0): A=\$950, B=\$2050

-> redo(T1): C=\$600

2. Checkpoints

A **checkpoint** is a mark point where all previous logs are saved permanently to the storage disk.

The **log files before the checkpoint** can be removed, and the **updated data** is stored safely on disk.

When a **checkpoint occurs**, all updates are written to the database, and unnecessary old logs are cleared.

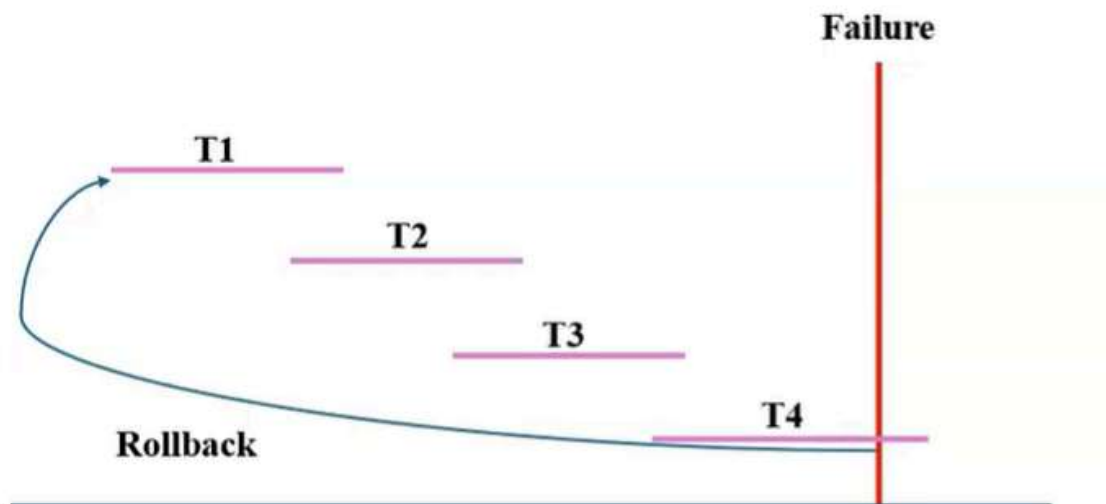


Without Checkpoint

When **checkpoint is not used**, the database must **search the entire log file from the beginning** after a failure.

If a system failure occurs:

- The database performs **rollback** of incomplete transactions.
- It must check **all previous transactions**, even very old ones.
- This makes recovery **slow** and **time-consuming**.



Suppose the following transactions are running:

- **T1** starts and runs
- **T2** starts after T1
- **T3** starts later
- **T4** starts just before failure
- Then **system failure occurs**

After failure:

- The database must **go back to the beginning**
- Perform **rollback** for incomplete transactions
- Much work is wasted.

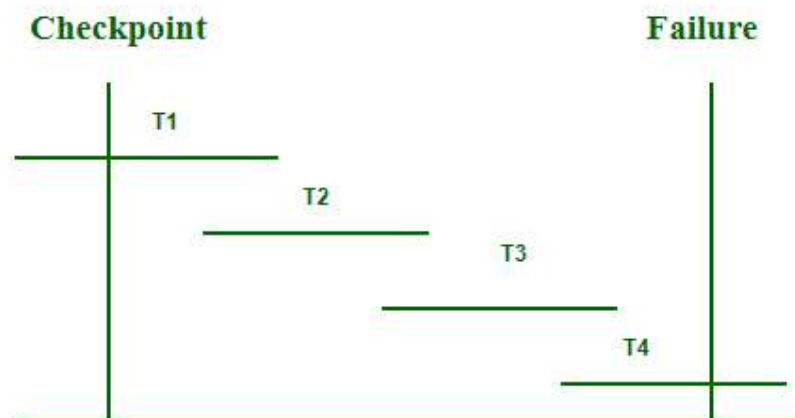
Demerits of Without Checkpoint

1. **Time Consuming**
Entire log file needs to be searched.
2. **Redoing Unnecessary Transactions**
Old transactions are checked again even if they were safe.

Steps to Use Checkpoints in the Database

- Write the begin_checkpoint record into a log.
- Collect checkpoint data in stable storage.
- Write the end_checkpoint record into a log.

The behavior when the system crashes and recovers when concurrent transactions are executed is shown below:



Understanding Checkpoints in multiple Transactions

Transactions and operations of the above diagram:

Transaction 1 (T1)	Transaction 2 (T2)	Transaction 3 (T3)	Transaction 4 (T4)
START			
	START		
	COMMIT		
		START	
		COMMIT	
			START
			FAILURE

Recovery Rule Summary

- **Redo List:** Transactions that have a COMMIT log (whether or not a START is shown).
- **Undo List:** Transactions that have a START but no COMMIT or ABORT i.e., incomplete transactions at the time of failure.

Transaction Analysis

T1:

- Has <START> but no COMMIT.
- Goes to Undo List.

T2:

- Has <START> and <COMMIT>.
- Goes to Redo List.

T3:

- Has <START> and <COMMIT>.
- Goes to Redo List.

T4:

- Has <START> but no COMMIT.
- System failed while T4 was running.
- Goes to Undo List.

Final undo and redo list :

- **Redo List:** T2, T3
- **Undo List:** T1, T4

All the transactions in the redo list are deleted with their previous logs and then redone before saving their logs. All the transactions in the undo list are undone and their logs are deleted.